

ÉPREUVE TECHNIQUE DE SÉLECTION

Architecture microservices, IA et développement full-stack

Rapport de réalisation

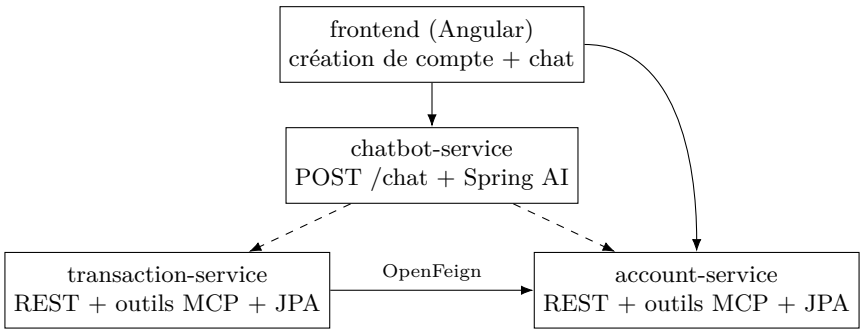
Services bancaires, assistant MCP et interface Angular

Youssef Rahioui

30 octobre 2026

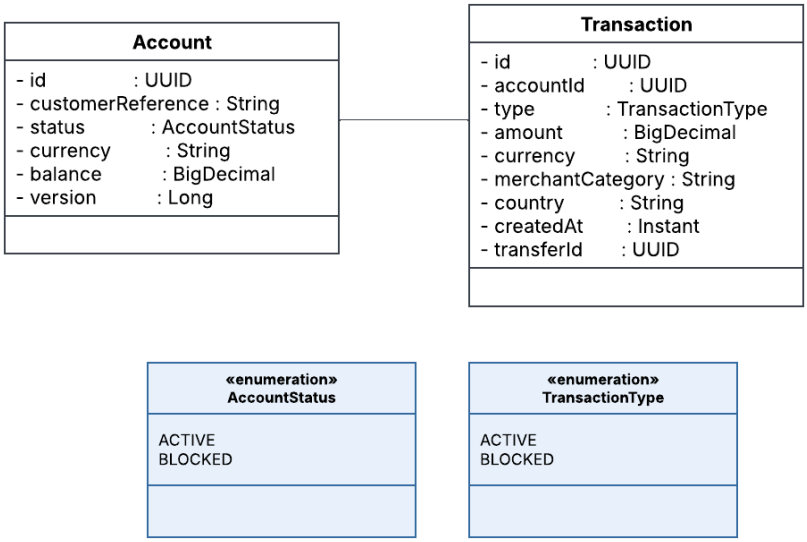
1. Périmètre et architecture

Le projet Maven comprend deux microservices métier, `account-service` et `transaction-service`, ainsi qu'un `chatbot-service`. Une application Angular se trouve dans `frontend/`. Chaque service métier possède une couche REST, un service applicatif et un dépôt JPA. `transaction-service` interroge `account-service` avec OpenFeign. L'assistant expose une API REST et est configuré pour appeler leurs outils MCP.



Traits pleins : appels HTTP (Angular vers comptes et chat, Feign entre services). Pointillés : connexions MCP du chatbot vers les outils métier.

Diagramme de classes fourni avec le projet. Le champ `Transaction.accountId` est une référence logique vers le compte, et non une association JPA entre les bases des deux services.



Précision : l'image indique ACTIVE/BLOCKED pour TransactionType ; dans le code, cette énumération vaut DEBIT/CREDIT. Le solde du compte est stocké en BigDecimal.

2. Fonctionnalités réalisées

Comptes. Création et consultation, lecture du solde, débit et crédit. Le service vérifie le statut actif, la devise, le montant positif et les fonds disponibles avant le débit ; la version JPA protège les mises à jour concurrentes.

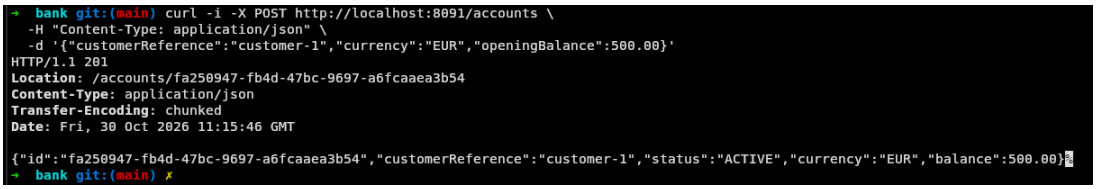
Transactions. Création, lecture, liste par compte et virement. Le plafond local est fixé à 1000 (paramètre `bank.transaction.max-amount`). Feign consulte le compte puis effectue débit ou crédit.

Assistant. `POST /chat` transmet le message à Spring AI (modèle configuré : `gpt-4o-mini`). Le client MCP référence les serveurs de comptes et de transactions ; les outils disponibles consultent comptes, soldes et transactions, et permettent de créer une transaction. La clé du modèle est lue depuis `OPENAI_API_KEY`.

3. Vérification et captures

`mvn test -q` passe sur les trois modules. Les tests couvrent les services métier, les contrôleurs REST et le chatbot avec dépendances simulées. Les captures ci-dessous documentent des appels locaux distincts : les identifiants des deux essais chatbot ne sont pas ceux d'une même séquence bancaire.

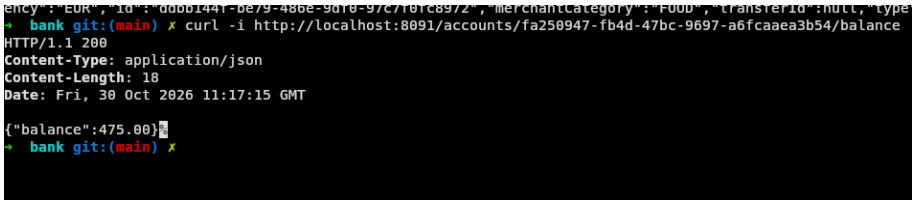
Essai REST des microservices. Un compte EUR est créé avec un solde de 500 ; une transaction de 25 est enregistrée ; le solde consulté est 475.



1. Création du compte — HTTP 201.

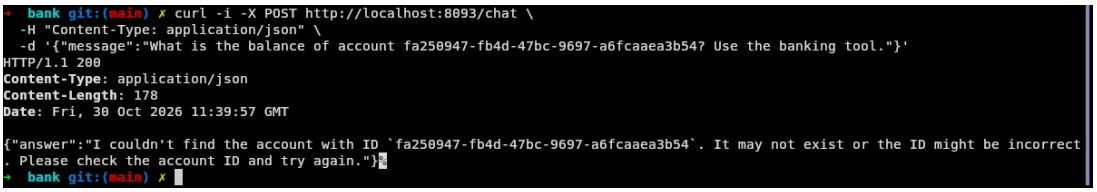


2. Transaction de 25 EUR — HTTP 201.

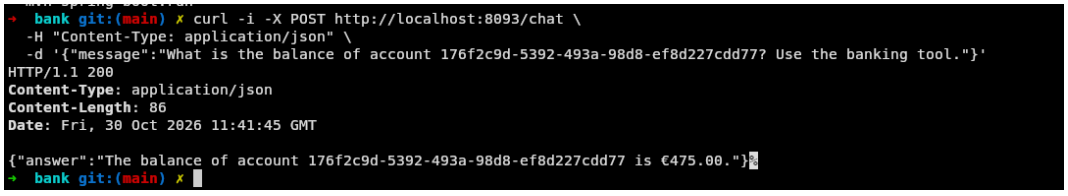


3. Solde après transaction : 475 EUR — HTTP 200.

Essais du chatbot via `POST /chat`. Les deux requêtes reçoivent une réponse HTTP 200. Pour un premier identifiant, l'assistant signale que le compte n'a pas été trouvé ; pour un second, il répond 475 EUR. Ces captures montrent des réponses de l'assistant, sans trace séparée de l'exécution d'un outil MCP.



4. Compte non trouvé : réponse du chatbot.

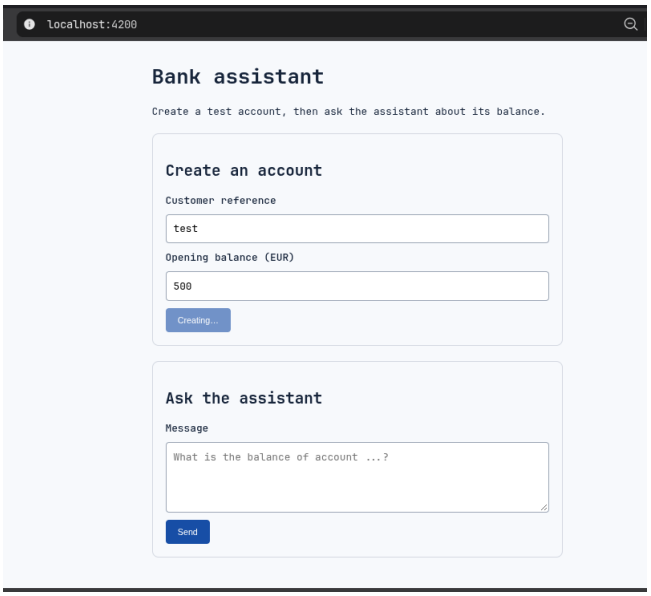


5. Consultation d'un autre compte : réponse de 475 EUR.

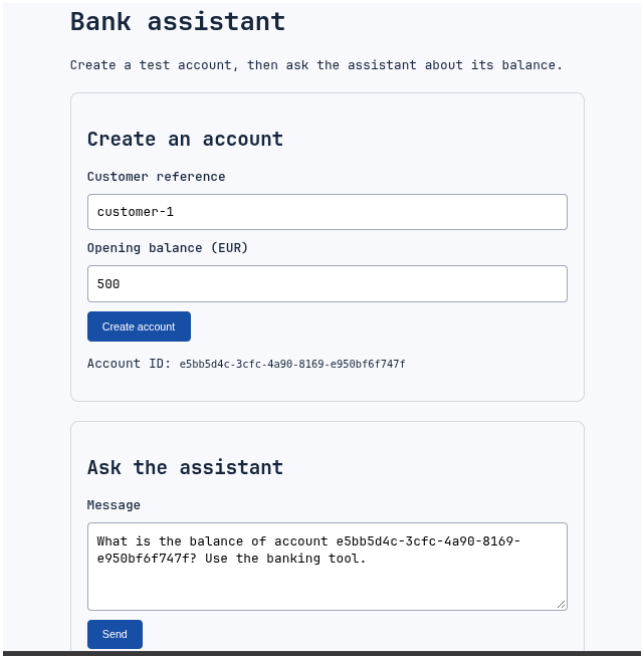
4. Interface web Angular

Le client `frontend/` permet de créer un compte EUR en saisissant une référence client et un solde initial. Après création, l'identifiant retourné est affiché et une question sur son solde est préparée automatiquement. Un second formulaire envoie la question à l'assistant et affiche sa réponse ou un message d'erreur.

En développement, le proxy Angular dirige `/accounts` vers le service de comptes (port 8091) et `/chat` vers le chatbot (port 8093). La compilation `npm run build` aboutit. La capture ci-dessous montre la page à `localhost:4200` pendant la création d'un compte. Les deux captures suivantes montrent un compte créé avec 500 EUR, puis une réponse de l'assistant affichant 500 EUR pour le même identifiant.



6. Interface pendant la création d'un compte.



7. Compte créé ; question préparée avec son identifiant.



8. Réponse du chatbot dans la page : solde 500 EUR.